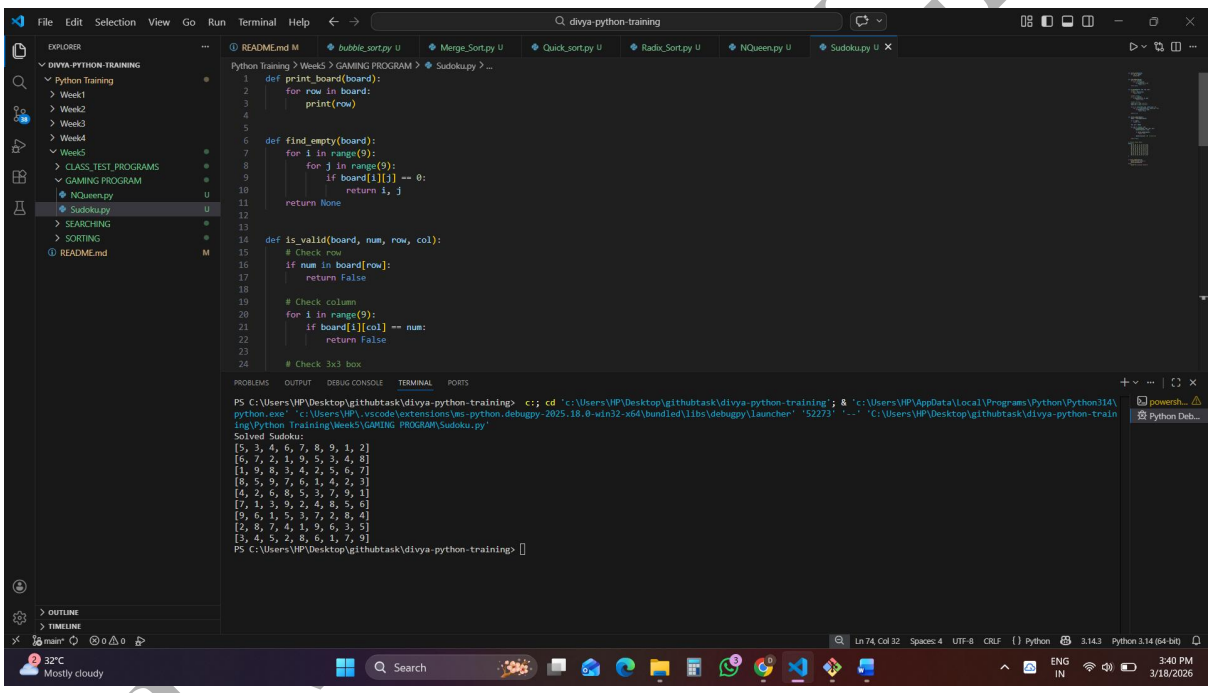
Sudoku :

```
def print_board(board):
    for row in board:
        print(row)

def find_empty(board):
    for i in range(9):
        for j in range(9):
            if board[i][j] == 0:
                return i, j
    return None

def is_valid(board, num, row, col):
    # Check row
    if num in board[row]:
        return False

    # Check column
    for i in range(9):
        if board[i][col] == num:
            return False

    # Check 3x3 box
    start_row = (row // 3) * 3
    start_col = (col // 3) * 3

    for i in range(start_row, start_row + 3):
        for j in range(start_col, start_col + 3):
            if board[i][j] == num:
                return False
```

DIVYA.G(III-IT)

```
    return True

def solve_sudoku(board):
    empty = find_empty(board)

    if not empty:
        return True

    row, col = empty

    for num in range(1, 10):
        if is_valid(board, num, row, col):
            board[row][col] = num

            if solve_sudoku(board):
                return True

            board[row][col] = 0  # backtrack

    return False

# Example Sudoku board
board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
```

DIVYA.G(III-IT)

```
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

if solve_sudoku(board):
    print("Solved Sudoku:")
    print_board(board)
else:
    print("No solution exists")
```